

NeoPager - LoRa-Based Bidirectional Wireless Pager System

1. Libraries Installed

The firmware depends on three third-party libraries, installed through the Arduino IDE's Library Manager (Sketch > Include Library > Manage Libraries, then searching each name and clicking Install), plus one library that ships with the board package itself and needs no separate installation.

Library	Author	Purpose in this project
RadioLib	Jan Gromes	Drives the SX1262 LoRa radio module - handles all packet transmission and reception.
U8g2	oliver	Drives the SSD1306 OLED display - all screens, fonts, and scrolling text.
Keypad	Mark Stanley & Alexander Brevig	Reads key presses from the 4x4 matrix membrane keypad.
Preferences	Built into the ESP32 core (Espressif)	Provides non-volatile flash storage, used to save the 15-message inbox history so it survives power loss. No separate installation required - it ships with the ESP32 board package.
Heltec_ESP32_LoRa_v3	Rop Gonggrijp (v0.9.2)	Heltec's own support library for the Heltec ESP32 LoRa v3 and Wireless Stick v3 boards. Installed alongside the board setup; the firmware in this project does not call into it directly.
LoRa	Sandeep Mistry (v0.8.0)	A widely-used Arduino LoRa library, primarily built for Semtech SX127x-series radios. It is installed in this environment but is not the library actually driving the radio in this project - RadioLib handles the SX1262 instead.

2. Board Manager Configuration

The Heltec WiFi LoRa 32 V3 is not included in the Arduino IDE by default. Support for it was added using Heltec's own board package

- Opened File > Preferences (Arduino IDE > Settings on some platforms).
- Added the following URL to the "Additional Boards Manager URLs" field:
https://resource.heltec.cn/download/package_heltec_esp32_index.json

- Opened Tools > Board > Boards Manager, searched for "heltec esp32", and installed the Heltec ESP32 Series board package.

3. Board Selected

Once the package was installed, the specific board and its settings were selected under Tools:

Setting	Value used
Board	Heltec WiFi LoRa 32(V3) - found under Tools > Board
Port	Port as per the device manager
Arduino IDE Version	---
CPU Frequency	240MHz (WiFi)
Core Debug Level	None
Erase All Flash Before Sketch Upload	Disabled
Events Run On	Core 1
Arduino Runs On	Core 1
LoRaWan DevEUI	CUSTOM
LoRaWan Preamble Length	8 (default)
LoRaWan Region	REGION_US915
LoRaWan Debug Level	None
Slow Clk Type (LoRaWAN only)	Internal (default)
Upload Speed	921600

Selecting this specific board entry (rather than a generic ESP32 or ESP32-S3 Dev Module) automatically configures the correct chip variant and default pin mappings for the Heltec V3 hardware.

Note: Since this project is intended for deployment in the **United States**, select **REGION_US915** under the **LoRaWAN Region** setting in the Arduino IDE. Do **not** select any other region.

Also in code the LORA_FREQ = 915MHz

4. COM Port Selection

Before uploading the firmware, connect the **Heltec WiFi LoRa 32 (V3)** board to your computer using a **USB-C cable**.

1. Connect the board to the PC.
2. The onboard **CP2102 USB-to-UART** interface will create a virtual COM port.
3. Open the **Arduino IDE** and go to **Tools → Port**.
4. Select the COM port corresponding to the Heltec board (for example, **COM3**, **COM7**, **COM13**, etc.). The assigned COM port may vary from one computer to another.
5. If you are unsure which port to select, open **Device Manager → Ports (COM & LPT)**. The board will appear as **Silicon Labs CP210x USB to UART Bridge** along with its assigned COM port number.
6. If no COM port appears, install the **Silicon Labs CP210x USB Driver**, reconnect the board, and check **Tools → Port** again.

Note: The COM port number is assigned automatically by your operating system and may be different each time the board is connected or when using a different computer. Always select the COM port associated with the **Silicon Labs CP210x USB to UART Bridge** device before uploading the firmware.

5. NeoPager Keys, Messages & How to Customize Them

The 4 Instant Messages (keys A, B, C, D)

These send immediately with a single key press , no typing, no confirmation needed.

Key	Message Sent
A	"I need help, please come now"
B	"I am at the meeting"
C	"Where are you?"
D	"Task done, acknowledged"

Press the key → it transmits right away → the sender sees "Sent Msg #X" → the receiver's buzzer beeps once and the message scrolls across their screen.

The 3 Editable / Template Messages (keys 1, 2, 3)

These are messages with a blank spot that gets filled in with a number you type before sending. Each one has a **fixed prefix**, a **number you enter**, and (sometimes) a **fixed suffix**.

Key	Prefix	You Type	Suffix	Final Message Example
1	"Will be back in "	a number	" min"	"Will be back in 20 min"
2	"Meet me at floor "	a number	(none)	"Meet me at floor 3"
3	"Call me in "	a number	" min"	"Call me in 10 min"

How to use them:

1. Press **1**, **2**, or **3** , the screen shows the template with __ where your number goes
2. Type digits on the keypad (up to 3 digits) they fill in live on screen
3. Press **#** to confirm and send
4. Press ***** at any point to cancel and go back

All Keys and What They Do

Key	Function
A / B / C / D	Send the matching instant message immediately
1 / 2 / 3	Start typing a template message
0	Open the Inbox (last 15 received messages)
4 (shown as 4^)	Scroll up through the inbox list
7 (shown as 7v)	Scroll down through the inbox list
#	Inside a template: confirm and send. Inside the inbox: view the selected message full-screen
*	Cancel/back exits template mode or exits the inbox back to Ready
5, 6, 8, 9	Currently unused free for future features (can send as numbers)

How to Change the Messages

Everything lives near the top of the .ino file, find and edit.

To change an instant message (A/B/C/D):

Find these four lines and just edit the text between the quotes:

```
const char* MSG_A = "I need help, please come now";
const char* MSG_B = "I am at the meeting";
const char* MSG_C = "Where are you?";
const char* MSG_D = "Task done, acknowledged";
```

For example, to change B to "Running late", just rewrite that line as:

```
const char* MSG_B = "Running late";
```

To change a template message (1/2/3):

Each template has two parts a **prefix** (before the number) and a **suffix** (after the number):

```
const char* TMPL_1_PRE = "Will be back in ";  
const char* TMPL_1_SUF = " min";  
const char* TMPL_2_PRE = "Meet me at floor ";  
const char* TMPL_2_SUF = "";  
const char* TMPL_3_PRE = "Call me in ";  
const char* TMPL_3_SUF = " min";
```

For example, if you wanted template 2 to say "Parking spot 5" instead of "Meet me at floor 3", change:

```
const char* TMPL_2_PRE = "Parking spot ";  
const char* TMPL_2_SUF = "";
```

Note : whatever you change, edit it identically on **both** boards , the message text itself isn't sent over LoRa for instant messages (only the key letter is sent, and each board looks up its own local text), so if the two boards have different text for the same key, they'll show different things. Same edit, both boards, every time.